



Prepared for academic submission

Project Type: Innovative Edge Computing System

Table of Contents

Table of Contents.....	2
Abstract.....	4
1. Introduction	4
2. Problem Statement	4
3. Objectives	4
4. Why Edge Computing is Important Here	5
5. System Overview	5
6. Proposed Architecture.....	6
7. Hardware Requirements	7
8. Software Requirements.....	8
9. Data Acquisition and Signal Processing	9
10. Machine Learning Model Design	9
11. Event Detection Logic	9
12. Communication Protocol	9
13. Alert Mechanism	10
14. Flow of Operation.....	10
15. Pseudocode	10
16. Testing and Evaluation.....	11
17. Example Test Cases	11
18. Security and Privacy Considerations.....	11
19. Limitations.....	12
20. Future Scope	12
21. Cost Estimate.....	12
22. Conclusion	12
References	13
23. Related Work and Background Study.....	13
24. Requirement Analysis	13
Functional Requirements.....	13
Non-Functional Requirements	13
25. Data Collection Strategy.....	14
26. Feature Engineering Pipeline	14
27. Model Training Procedure	15
28. Consensus and Voting Logic.....	15
29. Edge Deployment Details	16
30. Performance Considerations.....	16

31. Risk Analysis.....	17
32. User Interface and Dashboard	17
33. Experimental Setup.....	17
34. Sample Alert Policy	17
35. Project Timeline	17
36. Social and Practical Impact.....	18
37. Ethics and Responsible Design.....	18
38. Implementation Checklist	18
39. Appendix A: Sample MQTT Topics	18
40. Appendix B: Sample Event Record	19
41. Extended References.....	19
42. Literature Matrix	19
43. System Design Walkthrough	19
44. Message Formats and APIs	20
45. Failure Handling and Maintenance.....	20
46. Demonstration Plan.....	21
47. Deployment Recommendations	21
48. Final Remarks.....	21
49. Troubleshooting Guide	21
50. Glossary of Key Terms.....	22
51. Frequently Asked Questions	22
Q1. Why not send all sensor data to the cloud?.....	22
Q2. Why use multiple nodes instead of one?.....	22
Q3. Can this run without internet?	22
Q4. Is this ready for public safety deployment?	22
52. Short Project Summary	22
53. Suggested Source Folder Structure	23
54. Final Submission Checklist	23
Checklist.....	23
55. Closing Note	23

Abstract

Earthquakes are among the most dangerous natural disasters because they strike suddenly and leave very little time for evacuation or automatic response. Conventional early-warning systems often depend on centralized servers and cloud communication, which may introduce latency or failure when connectivity is poor. This report presents an Edge AI Network for Earthquake Early Alerts, a distributed architecture in which local edge devices analyze seismic signals near the source, share compact summaries with neighboring nodes, and collectively trigger alerts when confidence is high. The proposed system combines low-cost sensors, TinyML or lightweight machine-learning models, peer-to-peer verification, and optional cloud synchronization for reporting and monitoring. By moving inference closer to the sensor, the system reduces bandwidth usage, improves privacy, and remains operational even during internet outages.

Keywords: *edge computing, earthquake detection, TinyML, IoT, distributed systems, early warning, MQTT, seismic sensing.*

1. Introduction

Edge computing has become important wherever low latency, resilience, and data privacy matter. In a traditional cloud-centered design, sensor data is transmitted to a remote server, processed there, and then sent back as a decision or alert. That workflow can be too slow for emergency scenarios. Earthquake detection is a particularly demanding problem because the first few seconds after seismic activity begins are often the only window available for automated warnings. If the system waits for cloud confirmation, the warning may arrive too late.

An edge-based design addresses this limitation by performing local intelligence at or near the sensor. Each node can analyze vibration data in real time, distinguish normal disturbances from earthquake-like patterns, and exchange only short confidence signals with neighboring nodes. The architecture proposed in this report is designed to be practical, scalable, and suitable for college projects, hackathons, and research prototypes.

The main goal is not just detection, but dependable detection. A single false alarm can cause panic, while a missed alarm can be disastrous. Therefore, the system uses collaborative verification so that several nearby devices must agree before a high-priority alert is sent.

2. Problem Statement

Design a distributed earthquake early-warning system that uses edge devices to detect seismic vibrations locally, reduce false positives, and issue alerts quickly even when the internet is unavailable.

3. Objectives

- Detect abnormal vibrations locally at the sensor node.
- Use a lightweight AI model suitable for edge deployment.
- Verify detections using nearby nodes instead of relying on a single device.
- Minimize latency between event detection and alert generation.

- Reduce bandwidth consumption by avoiding continuous raw-data uploads.
- Preserve privacy by keeping raw sensor data on the edge where possible.
- Support offline operation during network outages or disasters.

4. Why Edge Computing is Important Here

Earthquake alerts are time-critical. The distance between a sensor, a cloud server, and the end user matters. In a cloud-only system, every vibration sample must travel to a remote data center before a decision is made. In contrast, edge computing lets the sensor node decide immediately whether the vibration is suspicious. That decision can then be shared locally with peer devices.

This approach has five major benefits. First, latency is lower because computation happens close to the data source. Second, reliability improves because the system can keep working even when the internet is down. Third, bandwidth consumption drops because only summaries and alerts are exchanged. Fourth, privacy improves because raw data stays near the source. Fifth, the architecture scales well because many small nodes can be added without overloading a central server.

5. System Overview

The proposed system consists of multiple sensing stations deployed across a city, campus, industrial area, or disaster-prone region. Each station includes a vibration sensor and an edge device. The edge device runs signal preprocessing and inference locally. If a node detects a possible earthquake, it publishes a compact event message using a lightweight protocol such as MQTT. Nearby nodes compare the event against their own readings. If multiple nodes observe similar abnormal vibration patterns within a short time window, the consensus engine raises the alert level.

The system can also forward a summarized event to a cloud dashboard. The cloud is optional and is used only for logging, analytics, mapping, and long-term historical tracking.

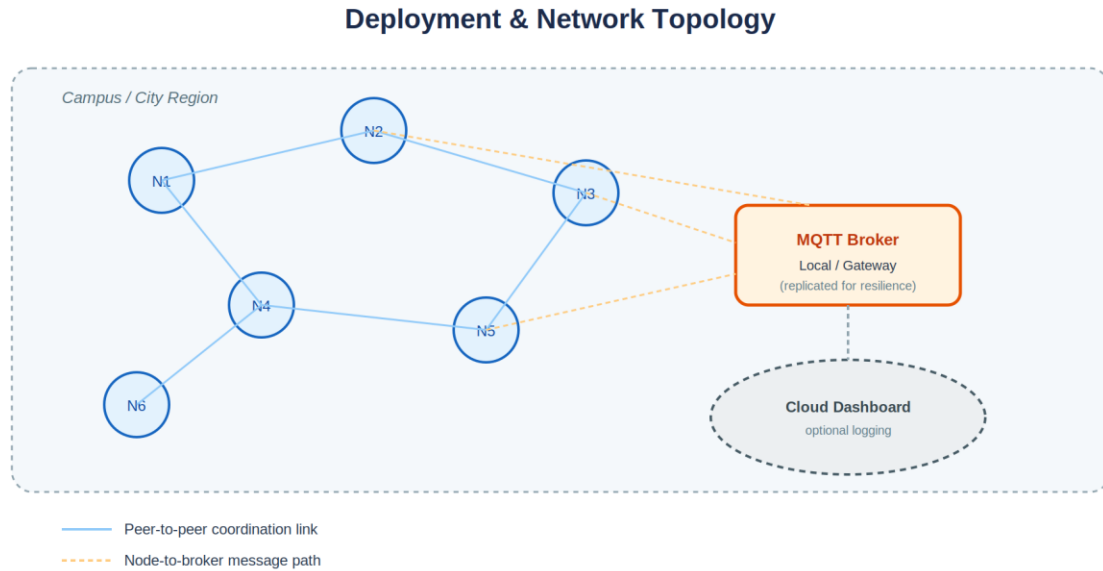


Figure 1. Deployment and network topology showing peer-to-peer coordination links and the optional broker/cloud path.

6. Proposed Architecture

The architecture can be understood in five layers:

- Sensing layer: collects raw vibration or acceleration signals from low-cost sensors.
- Edge processing layer: filters noise, extracts features, and runs the ML model locally.
- Coordination layer: exchanges confidence messages with neighboring edge nodes.
- Decision layer: applies voting or consensus logic to decide whether an event is real.
- Alert layer: activates sirens, SMS, mobile notifications, dashboards, or gateway alarms.

Five-Layer Edge AI Architecture

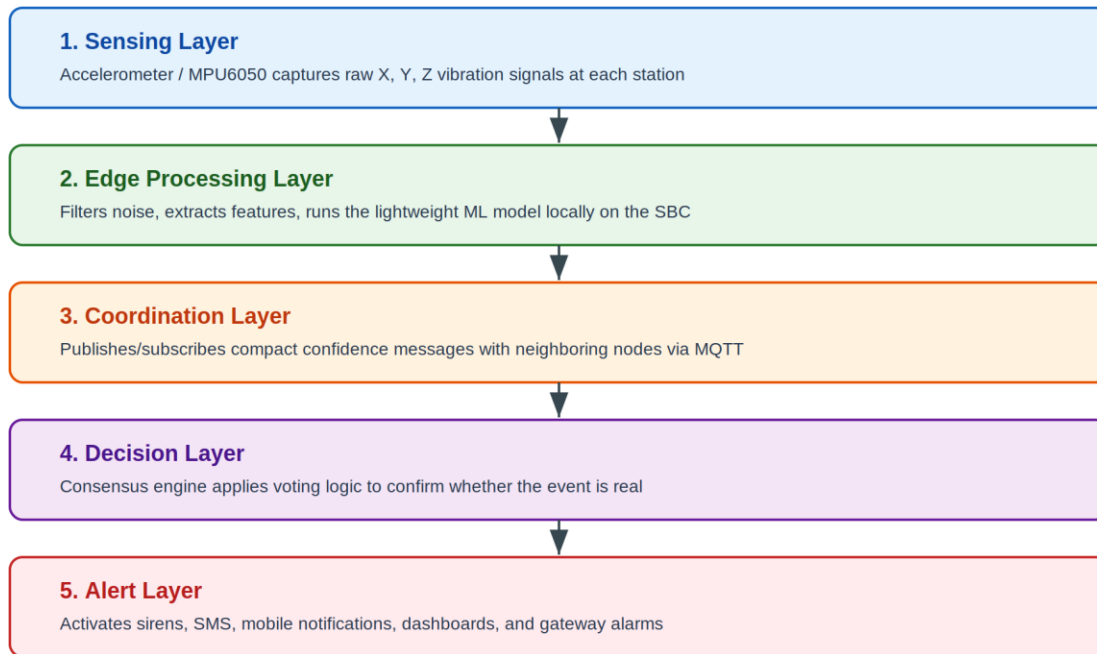


Figure 2. The five-layer Edge AI architecture, from raw sensing to final alert.

A representative data flow is: Sensor -> Edge Device -> Local Inference -> Neighbor Verification -> Consensus -> Alert.

End-to-End Event Flow

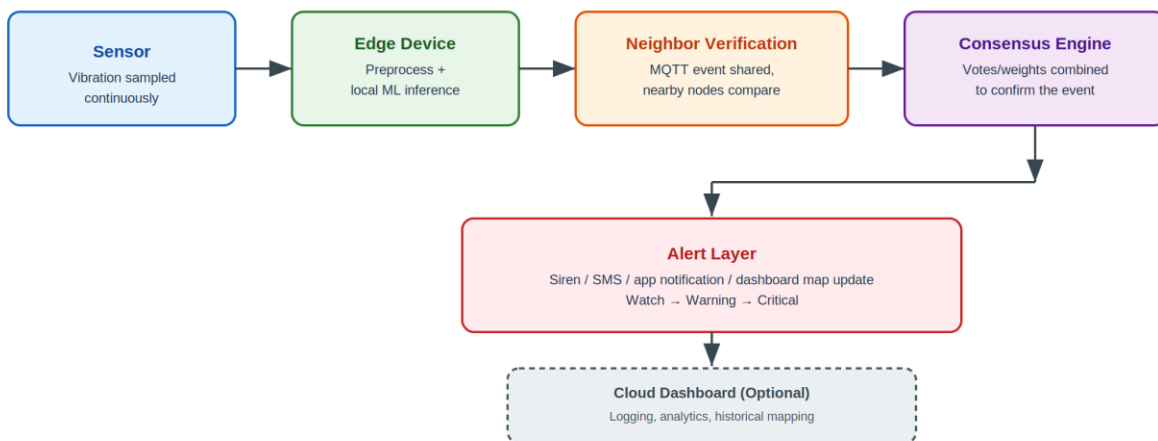


Figure 3. End-to-end event flow across the architecture layers.

7. Hardware Requirements

Component	Purpose	Suggested Option
Edge device	Runs local AI and communication stack	Raspberry Pi 5 / Jetson Nano / similar SBC
Motion sensor	Captures seismic vibrations	MPU6050 / accelerometer module
Location module	Tags event location	GPS module
Alert unit	Provides audible or electrical warning	Buzzer / relay / siren
Communication	Sends compact messages	Wi-Fi / Ethernet / GSM / LoRa
Power	Keeps node active	Adapter / battery backup / solar

Edge Node — Hardware Block Diagram

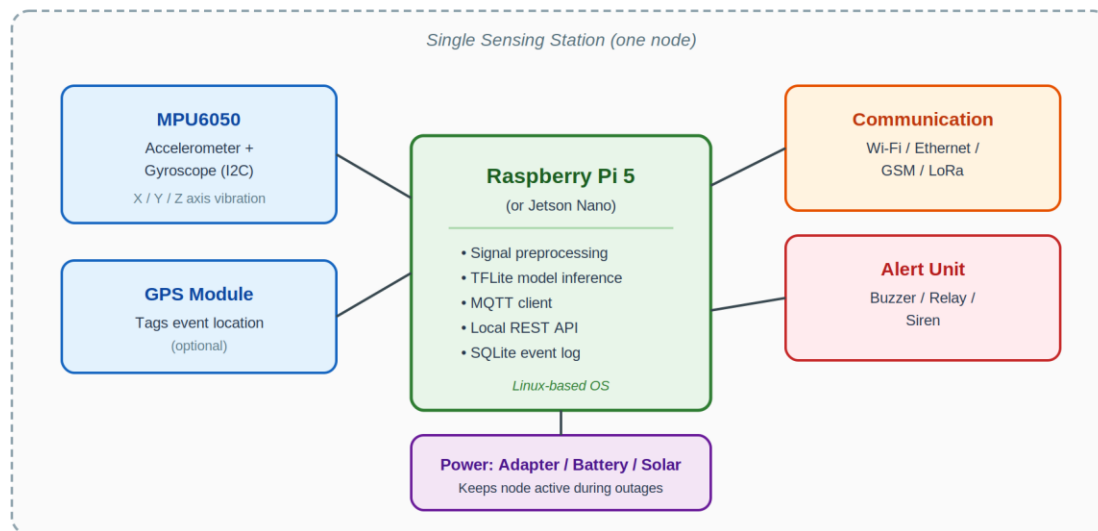


Figure 4. Hardware block diagram for a single sensing station (one node).

8. Software Requirements

- Python for core logic and data processing.
- TensorFlow Lite or another lightweight inference framework.
- MQTT for node-to-node messaging.
- FastAPI or Flask for local monitoring APIs.
- SQLite for local event logs.
- Docker for deployment and reproducibility.
- Linux-based operating system on the edge device.

9. Data Acquisition and Signal Processing

The raw signal from an accelerometer is noisy and must be processed before classification. The system can sample acceleration values on the X, Y, and Z axes at a fixed frequency. A short sliding window, for example 1 to 3 seconds, is maintained for analysis. Within each window, the software may remove spikes, normalize the signal, and compute derived features.

Common feature candidates include mean acceleration, standard deviation, maximum amplitude, root mean square, zero-crossing rate, dominant frequency, and spectral energy. These features help the model distinguish earthquakes from ordinary disturbances such as footsteps, traffic, machinery, and handheld vibrations.

10. Machine Learning Model Design

Several model families are suitable depending on the available data and device capability. A 1D Convolutional Neural Network can learn directly from time-series segments. An LSTM model is useful if the temporal order of vibrations is important. Classical models such as Random Forest or XGBoost can also work well when feature engineering is strong and compute budget is limited.

For a student project, a strong practical approach is to start with feature extraction plus Random Forest, then compare it with a lightweight 1D CNN. If the CNN performs better and remains small enough, it can be converted to TensorFlow Lite for edge deployment.

Model	Strength	Limitation
Random Forest	Simple, fast, interpretable	May miss complex temporal structure
XGBoost	Excellent tabular performance	Needs careful tuning
LSTM	Learns time dependencies	Heavier than tree-based models
1D CNN	Good for signal patterns	Needs training data and tuning

11. Event Detection Logic

A good detection pipeline includes multiple stages. First, the node detects whether the vibration is unusual. Second, it estimates confidence. Third, it sends a short message containing the timestamp, location, confidence score, and event category. Fourth, neighboring nodes compare this information with their own sensor observations. Fifth, the system decides whether the abnormal signal is local noise or a broader seismic event.

A simple rule-based consensus could require at least three nodes within a region to report similar anomalies within a defined time window. A more advanced version could use weighted voting, where nodes closer to the epicenter contribute more heavily if their readings are stronger.

12. Communication Protocol

MQTT is a suitable protocol because it is lightweight and supports publish-subscribe communication. Each node can publish on topics such as seismic/node01, seismic/alerts, or seismic/regionA. The broker can run locally or on a nearby gateway. In a disaster setting, the broker itself may be replicated so the system does not fail if one server stops.

For peer verification, nodes do not need to send raw vibration streams. Instead, they share compact metadata such as confidence score, window index, intensity level, and sensor health status. This keeps traffic low and the network responsive.

13. Alert Mechanism

Once consensus is reached, the alert layer can activate multiple outputs. A local buzzer can warn nearby people immediately. A relay can connect to building sirens. SMS or mobile app notifications can be sent to registered users, emergency teams, and administrators. A dashboard can also display the active alert on a map for command-center visibility.

The alert logic may support levels such as Watch, Warning, and Critical. Watch indicates suspicious activity under observation. Warning means the system sees a likely event. Critical indicates that multiple nodes have confirmed the event and emergency response should begin.

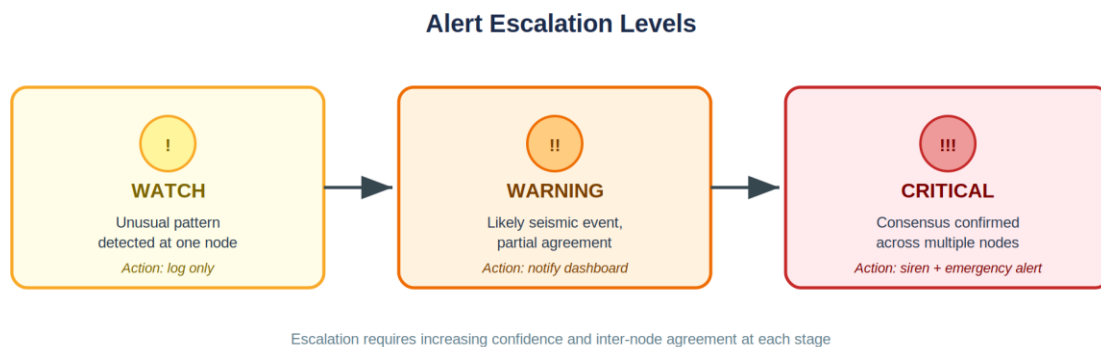


Figure 5. Alert escalation levels from Watch to Critical.

14. Flow of Operation

1. Sensor continuously samples ground vibration.
2. Edge device preprocesses the signal in real time.
3. ML model estimates whether the vibration is abnormal.
4. Node publishes a short event summary if confidence is high.
5. Neighboring nodes compare their own observations.
6. Consensus engine aggregates the evidence.
7. Alert system triggers siren, notification, and dashboard update.
8. Optional cloud logs the event for later analysis.

15. Pseudocode

```

while True:
    data = read_sensor_window()
    features = preprocess(data)
    score = model.predict(features)
    if score > threshold:
        publish_event(score, timestamp, location)
        replies = collect_neighbor_responses(time_window=short)
        if consensus(replies) >= required_votes:
            trigger_alert(level='CRITICAL')
        else:
            mark_as_possible_false_alarm()

```

16. Testing and Evaluation

Testing should evaluate both model accuracy and system performance. Accuracy measures whether earthquake-like events are classified correctly. System performance measures latency, packet usage, CPU load, and reliability.

Metric	What It Measures	Target
Accuracy	Correct classification	High
Precision	False alarm control	High
Recall	Missed event reduction	Very high
Latency	Time from vibration to alert	Low
Bandwidth	Amount of transmitted data	Low
Uptime	Availability during outage	High

17. Example Test Cases

Test Case	Input Scenario	Expected Result
TC1	Normal walking near sensor	No alert
TC2	Passing truck on road	No critical alert
TC3	Strong local vibration at one node only	Possible false alarm, no consensus
TC4	Earthquake-like signal on multiple nodes	Critical alert
TC5	Internet disconnected	Local detection still works

18. Security and Privacy Considerations

Because the system may be deployed in public or sensitive locations, security matters. Nodes should authenticate messages so malicious devices cannot inject fake alerts. MQTT topics should be protected using secure credentials where possible. The system should also log sensor health and communication failures to help detect tampering or device malfunction.

Privacy is protected because raw sensor streams do not need to leave the device. Only event summaries and confidence scores are shared. This is a strong advantage over camera-heavy or cloud-only systems.

19. Limitations

- The quality of detection depends on sensor calibration and placement.
- The system needs representative training data from real and simulated vibrations.
- Dense deployment may be required for better geographic coverage.
- Battery-backed operation becomes important in power failures.
- Communication delays between nodes may affect consensus timing.

20. Future Scope

Future versions can support federated learning so each edge node improves the model without uploading raw data. The system can also integrate with drones, CCTV, traffic lights, railway signals, and disaster management centers. Another enhancement is aftershock prediction using historical patterns and multi-sensor fusion.

A more advanced research direction is adaptive edge swarm intelligence, where devices dynamically decide whether to process locally, share with neighbors, or send summary data to the cloud depending on urgency and bandwidth.

21. Cost Estimate

Item	Approximate Cost (INR)	Note
Raspberry Pi / SBC	5000-9000	Depends on model
Accelerometer sensor	150-500	Low-cost IoT sensor
GPS module	300-800	Optional
Alert hardware	200-1000	Buzzer / relay / siren
Power and casing	500-2000	Enclosure and backup

22. Conclusion

The proposed earthquake early warning system demonstrates how edge computing can solve real-time, safety-critical problems. By keeping inference local and using cooperative verification, the system reduces latency, improves reliability, and continues functioning during connectivity

failures. The design is practical enough for a prototype and rich enough for a final-year or research project.

References

1. IEEE literature on earthquake early warning systems and seismic sensing.
2. Documentation for TensorFlow Lite and lightweight on-device inference.
3. MQTT specification and publish-subscribe messaging references.
4. Raspberry Pi and embedded Linux documentation.
5. IoT and edge computing survey papers for architectural background.

23. Related Work and Background Study

Research on earthquake early warning has traditionally focused on sensor networks, signal analysis, and central decision systems. Many academic and industrial solutions rely on dense arrays of sensors that feed a central server, where the data is fused and interpreted. That approach is effective when the communication network is stable, but it may fail when infrastructure is damaged by the same event it is trying to detect.

A second stream of work has explored low-power embedded devices and TinyML. These systems demonstrate that useful classification can happen on microcontrollers or compact single-board computers. For earthquake detection, this is important because vibration signatures can often be learned from short time windows using lightweight models. The challenge is not merely achieving high accuracy, but doing so with predictable latency and minimal power consumption.

A third area of interest is distributed consensus. In safety systems, a single sensor should not always be trusted blindly. Combining local inference with neighborhood verification creates a more robust design. This is the main novelty of the proposed project: instead of a cloud-first architecture, the system is organized as a network of cooperative edge nodes.

24. Requirement Analysis

Before implementation, the system requirements should be divided into functional and non-functional categories. Functional requirements describe what the system must do, while non-functional requirements describe how well it must do it.

Functional Requirements

- Read vibration samples continuously from the sensor.
- Classify signal windows as normal or abnormal.
- Send an event summary to neighboring nodes.
- Generate alerts when consensus is achieved.
- Store local logs for later review.

Non-Functional Requirements

- Keep latency low enough for real-time response.
- Remain stable under intermittent connectivity.
- Minimize power usage and data transfer.
- Support simple deployment on low-cost hardware.

25. Data Collection Strategy

The quality of the detection model depends heavily on the dataset. Since real earthquake datasets may be limited or hard to collect at the project stage, a practical strategy is to combine multiple sources of data. Real seismic records can be used where available, while simulated and proxy vibration data can supplement the training process. For example, walking, vehicle movement, hammering, small machine vibration, and table knocks can be recorded as non-earthquake classes. Earthquake-like motion can be represented using labeled seismic recordings from public datasets or controlled shake-table experiments if hardware is available.

Every sample should include a timestamp, sensor ID, location tag, sampling rate, and class label. Balanced class distribution is important because a model trained on too many non-earthquake examples and too few earthquake examples may become biased. If the classes are imbalanced, techniques such as resampling, class weights, or synthetic augmentation can be used.

Data Type	Example Source	Purpose
Real seismic records	Public earthquake datasets	Training and validation
Traffic vibrations	Roadside sensor logging	Negative class
Footstep vibrations	Indoor sensor recording	Negative class
Impact events	Knocks, bangs, machinery	Hard negative examples
Synthetic windows	Signal augmentation	Model robustness

26. Feature Engineering Pipeline

Signal preprocessing transforms raw accelerometer data into features that the model can learn from. One common approach is to segment the stream into overlapping windows. Each window is cleaned using a simple filter, normalized, and then converted into either handcrafted features or a direct sequence input.

- Window size: 1 second to 3 seconds depending on sampling frequency.
- Overlap: 25% to 50% to avoid missing short events.
- Filtering: moving average or band-pass filtering to reduce noise.
- Normalization: scale values so the model is less sensitive to device-specific bias.
- Feature extraction: RMS, amplitude, variance, skewness, kurtosis, peak count, and spectral energy.

Handcrafted features are useful when computation is limited. Raw sequence input is useful when the model should learn patterns directly. A student project can implement both methods and compare them experimentally.

27. Model Training Procedure

The model training pipeline begins by splitting the dataset into training, validation, and test sets. The training set teaches the model the general patterns of seismic and non-seismic motion. The validation set is used to tune hyperparameters and choose the best model. The test set is reserved for final evaluation and must not influence training decisions.

1. Load labeled vibration windows.
2. Apply preprocessing and feature extraction.
3. Split into train/validation/test subsets.
4. Train a baseline model.
5. Evaluate precision, recall, F1-score, and confusion matrix.
6. Tune thresholds or hyperparameters.
7. Export the best lightweight model to the edge device.

For deployment, model compression matters. A large model may perform well on a desktop but become too slow on a Raspberry Pi. Therefore, the final step should measure inference time on the target hardware, not just accuracy on a workstation.

28. Consensus and Voting Logic

The consensus layer is what makes the system innovative. A node does not need to decide everything on its own. Instead, it can ask nearby nodes whether they have observed similar anomalies. Several strategies are possible.

- Majority vote: if most nearby nodes report abnormal activity, raise the alert.
- Weighted vote: assign more importance to nodes with stronger or earlier readings.
- Confidence threshold: combine model scores and require a minimum average confidence.
- Time-window agreement: only consider nodes that report within a short interval.

This consensus logic reduces false positives from local disturbances. A truck passing near one sensor may trigger one node, but neighboring sensors may remain normal. An actual seismic event is more likely to appear across multiple nodes in the same region.

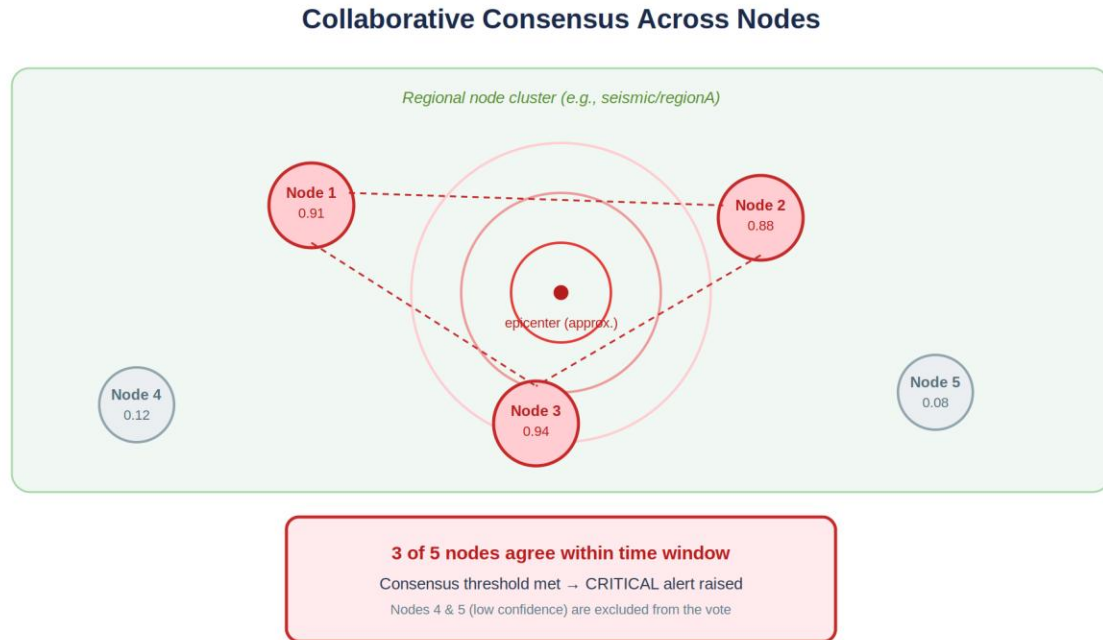


Figure 6. Example of collaborative consensus: three nearby nodes agree within the time window while two distant nodes do not, confirming a regional event.

29. Edge Deployment Details

A real deployment would typically run a lightweight service on each node. One process reads the sensor and stores samples in a ring buffer. A second process performs inference. A third service handles MQTT messaging and alert publication. A fourth component exposes a local dashboard or health endpoint.

Module	Responsibility	Runs On
Sensor daemon	Reads accelerometer data	Edge device
Inference engine	Predicts event class	Edge device
Message broker client	Publishes and subscribes alerts	Edge device/gateway
Alert controller	Activates buzzer or siren	Edge device
Logging service	Stores event history	Edge device/cloud optional

30. Performance Considerations

Performance should be assessed from the user and system perspectives. From a user perspective, the most important question is how quickly the warning reaches people. From a system perspective, the key question is how much CPU, memory, and energy the edge node consumes. The design should therefore favor compact models, efficient window sizes, and low-overhead messaging.

If the model is too heavy, it may miss the time window in which an alert is valuable. If the consensus logic is too strict, it may delay the alert. If the threshold is too loose, false alarms may occur. The best design balances these trade-offs.

31. Risk Analysis

There are several technical and practical risks. Sensor noise can create false triggers. Poor calibration can shift readings across devices. Connectivity failures can prevent neighboring nodes from exchanging summaries. Low battery or power loss can disable a station. To reduce these risks, the project should include health monitoring, device-level logging, and fallback alert rules.

- Risk: noisy environment. Mitigation: filtering and feature engineering.
- Risk: device failure. Mitigation: redundant nodes and heartbeat checks.
- Risk: connectivity loss. Mitigation: offline local detection and delayed sync.
- Risk: model drift. Mitigation: periodic retraining and threshold review.

32. User Interface and Dashboard

A simple dashboard can make the project feel complete and demo-ready. It may show sensor status, last detected event, region-wise confidence, and alert history. A map-based interface is especially useful when multiple nodes are distributed across a campus or city. The dashboard does not need to process raw data; it only needs to present summaries in a clean and actionable way.

33. Experimental Setup

For a college prototype, experiments can be run in a small indoor lab or classroom environment. Multiple edge nodes can be placed at different positions. Various controlled vibration sources can be introduced, such as footsteps, table taps, and a simulated vibration source. The system can then be evaluated for response time, false alarms, and detection rate.

A useful experiment is to compare a single-node detector against the collaborative multi-node version. This demonstrates the value of edge collaboration, not just edge inference.

34. Sample Alert Policy

Level	Meaning	Action
Watch	Unusual pattern detected	Log only
Warning	Likely seismic event	Notify local dashboard
Critical	Consensus confirmed	Activate siren and emergency alert

35. Project Timeline

Week	Task
1	Study related work and finalize architecture
2	Collect sample vibration data
3	Build preprocessing pipeline
4	Train baseline model
5	Convert model for edge deployment
6	Implement MQTT messaging
7	Create consensus logic and alert layer
8	Build dashboard and finalize report

36. Social and Practical Impact

If deployed in schools, railway stations, factories, or residential areas, this system could help reduce panic and response delay. Even a small warning window can be valuable for automated shutdowns, elevator safety, train control, and public awareness. The design also has educational value because it introduces students to real-world intersection of IoT, AI, and distributed computing.

37. Ethics and Responsible Design

A disaster-warning system must be designed carefully because false confidence can be harmful. The project should clearly communicate that it is a prototype and not a certified public safety system. Any deployment should be tested with experts, calibrated properly, and validated against real-world requirements before operational use.

38. Implementation Checklist

- Acquire and test sensor hardware.
- Create data logging format.
- Implement preprocessing and feature extraction.
- Train baseline classifier.
- Benchmark latency on edge hardware.
- Add MQTT message exchange.
- Add consensus engine.
- Connect alert hardware and dashboard.
- Write test cases and final documentation.

39. Appendix A: Sample MQTT Topics

```
seismic/node01/status
seismic/node01/event
seismic/node01/heartbeat
seismic/regionA/summary
seismic/alerts/critical
```

40. Appendix B: Sample Event Record

```
{ timestamp: '2026-06-20T10:15:30', node_id: 'node01', confidence: 0.92, location:
'Campus Block A', severity: 'Warning' }
```

41. Extended References

- 6. Survey papers on TinyML for sensor-based classification.
- 7. Research on distributed consensus in IoT and edge systems.
- 8. Documentation on Python signal processing libraries.
- 9. Open-source repositories demonstrating MQTT-based sensor networks.
- 10. Articles on disaster resilience and early warning system design.

42. Literature Matrix

To clarify the novelty of the project, it helps to compare the proposed design with common alternatives. The table below summarizes the main differences in architecture and execution style.

Approach	Processing Location	Strength	Weakness
Cloud-only seismic system	Remote server	Centralized analytics	High latency and dependency on connectivity
Single edge sensor	Local device	Fast detection	Limited robustness against false alarms
Proposed cooperative edge network	Distributed edge nodes	Fast and resilient	Requires coordination logic

The cooperative edge network is the best balance for a real-time warning scenario because it combines local speed with distributed verification. It is more reliable than a single local sensor and more responsive than a cloud-centered workflow.

43. System Design Walkthrough

The complete flow begins when the accelerometer senses movement. A sampling service stores the latest readings in a short buffer. A preprocessing routine smooths the data and prepares it for inference. The classifier assigns a probability score to the current window. If the score crosses a threshold, the device creates a compact event message and publishes it to a regional topic.

Neighbor nodes read the event, compare it against their own local readings, and respond with either confirmation or rejection. The consensus engine then decides whether enough evidence exists to escalate the event. This layered approach ensures that a local disturbance does not automatically trigger a citywide alarm.

A simple but effective design rule is to keep the time window short enough for quick response yet long enough to capture the seismic pattern. This trade-off is one of the most important design choices in the project.

44. Message Formats and APIs

Compact message formats are useful because they reduce network traffic and simplify debugging. A recommended event payload can include the node identifier, timestamp, estimated severity, confidence score, and battery status.

```
{'node_id': 'node01', 'timestamp': '2026-06-20T10:15:30', 'confidence': 0.92, 'severity': 'Warning', 'battery': 87}
```

For the local dashboard, a small REST API can expose health and event status. Example endpoints include GET /health for node health, GET /events for recent events, and GET /status for the current alert level. These endpoints make integration easy without requiring heavy infrastructure.

Endpoint	Method	Purpose
/health	GET	Node health and uptime
/events	GET	Recent seismic events
/status	GET	Current alert level
/calibrate	POST	Trigger sensor calibration

45. Failure Handling and Maintenance

A strong edge system must handle failures gracefully. If the sensor becomes disconnected, the node should mark itself unhealthy and alert the dashboard. If the MQTT broker becomes unavailable, the node can buffer the last few events locally and retry later. If a node restarts unexpectedly, it should announce a heartbeat message so the network knows it is alive again.

- Heartbeat messages confirm node availability.
- Watchdog timers help restart stalled services.
- Local logs provide evidence for debugging and audits.

- Calibration routines help correct drift over time.
- Battery monitoring supports proactive maintenance.

46. Demonstration Plan

For presentation or viva purposes, the project can be demonstrated in a staged sequence. First, show the dashboard with all nodes healthy. Second, simulate a light vibration and show that the system logs the event without a critical alert. Third, trigger a stronger and more sustained vibration across multiple nodes. Finally, show how the system reaches consensus and activates the alert mechanism.

This demo flow is effective because it clearly shows the difference between ordinary noise and a real event. It also proves the value of collaborative verification rather than relying on one sensor alone.

47. Deployment Recommendations

In a real deployment, the nodes should be installed at structurally strategic points such as building corners, floor levels, bridge supports, or street intersections. The placement should maximize coverage while minimizing false triggers from machine vibration or human activity. The network should also include redundancy so that a few failed nodes do not disable the whole area.

For public safety use, the system would need calibration, environmental testing, and expert review. The current project is best understood as a prototype architecture that demonstrates the edge-computing concept clearly.

48. Final Remarks

This project is suitable for a final-year submission because it includes sensing, machine learning, edge deployment, distributed communication, consensus logic, and emergency alerts. It is also innovative because it does not rely on a single cloud server to make the critical decision. Instead, it uses a network of collaborative edge devices that can continue operating during outages.

With a well-built prototype, this idea can be extended into disaster management, smart city infrastructure, or research on cooperative edge intelligence.

49. Troubleshooting Guide

During development, the most common issues are sensor noise, serial read errors, delayed alerts, and connection failures. A structured troubleshooting guide helps the team fix problems quickly during a demo or field test.

- If sensor values stay constant, check wiring and power supply.
- If alerts do not appear, verify the threshold and consensus logic.
- If MQTT messages are missing, confirm broker address and topic names.

- If inference is slow, reduce window size or use a smaller model.
- If the node reboots unexpectedly, inspect memory usage and power stability.

50. Glossary of Key Terms

Term	Meaning
Edge Computing	Processing data near the source instead of a remote cloud server.
TinyML	Machine learning on very small or resource-constrained devices.
Consensus	Agreement among multiple nodes before an alert is issued.
Latency	Time delay between sensing an event and responding to it.
MQTT	Lightweight messaging protocol for IoT systems.
Inference	Using a trained model to make a prediction on new data.
False Positive	A normal event incorrectly classified as an earthquake.
False Negative	A real earthquake missed by the system.

51. Frequently Asked Questions

Q1. Why not send all sensor data to the cloud?

Because that would increase latency and bandwidth usage. In an emergency system, the decision must happen as close to the sensor as possible.

Q2. Why use multiple nodes instead of one?

Multiple nodes reduce the chance of false alarms and make the system more reliable when one sensor is noisy or faulty.

Q3. Can this run without internet?

Yes. Local detection and local consensus can keep working offline. Internet is optional for logging and remote monitoring.

Q4. Is this ready for public safety deployment?

No. It is a prototype architecture for academic and research use. Real deployment would require certification, expert validation, and extensive field testing.

52. Short Project Summary

The project builds a distributed earthquake warning system using edge AI, local vibration sensing, peer verification, and low-latency alerts. It demonstrates how edge computing can make a safety-critical application faster, more reliable, and more resilient than a cloud-only design.

53. Suggested Source Folder Structure

A clean codebase makes the project easier to understand and present. The structure below separates sensor handling, models, messaging, and dashboard code so that each part can be developed independently.

Folder/File	Purpose
src/sensors/	Sensor reading and calibration code
src/preprocess/	Filtering and feature extraction
src/model/	Training, evaluation, and inference
src/messaging/	MQTT publish-subscribe utilities
src/alerts/	Buzzer, siren, and notification logic
src/dashboard/	Web or local monitoring interface
data/	Collected and labeled sample data
models/	Saved trained model artifacts
docs/	Project report, diagrams, and notes

54. Final Submission Checklist

Before submission, the project should be checked for technical correctness, documentation quality, and presentation readiness.

Checklist

- Code runs without errors on the target device.
- Model file is saved and loads correctly.
- Sensor data is being read and logged properly.
- Alert trigger works in a controlled test.
- Consensus logic is demonstrated with at least two or three nodes.
- Report contains abstract, diagrams, methodology, results, and references.
- Slides summarize the architecture and demo flow clearly.

55. Closing Note

This expanded report is intentionally structured like a practical academic submission. It should be easy to trim for a short report or expand further into a final-year project document with figures, diagrams, and results.